

Technical Challenge — Senior Software Engineer

Context

This challenge requires the candidate to create a system to **search, register, and maintain genres, authors, and books**.

Beyond implementation, the goal is to assess the candidate's ability to **design a coherent technical solution, make well-supported decisions, and clearly explain the main trade-offs**.

This challenge is designed to be completed within **3 days**. A perfect or overly comprehensive solution is not expected. The expected outcome is a functional, well-organized solution supported by clear technical reasoning.

Objective

Develop an application consisting of:

- a **REST API in .NET/C#**;
- an **SPA application in Angular or React**;
- a **relational database**.

The application must allow the management of:

- genres;
- authors;
- books.

Business Rules

The application must follow these business rules:

- A genre may have multiple books.
- An author may have multiple books.
- Each book belongs to only one author.
- Each book belongs to only one genre.
- Users must be able to create, search, update, and delete genres, authors, and books.

The candidate may propose additional validations, provided they are consistent with the domain and explained in the documentation or presentation.

Database

The solution must use a relational database.

The candidate must choose one of the options below:

- SQL Server

- PostgreSQL
- MySQL

The selected database must be justified.

Backend

A **REST API using .NET/C#** must be developed.

The API must provide operations to create, read, update, and delete genres, authors, and books.

The solution must include:

- clear code organization;
- appropriate separation of responsibilities;
- proper error handling;
- consistent HTTP responses;
- persistence in a relational database;
- configuration for local execution;
- automated tests for the main scenarios.

No specific architecture is required. The candidate should choose the approach they consider most appropriate for the challenge and justify their decisions.

Frontend

An **SPA application using Angular or React** must be developed to consume the API.

The application must allow users to:

- list genres, authors, and books;
- register records;
- edit records;
- remove records;
- view the relationship between book, author, and genre.

The interface does not need to be visually sophisticated, but it must be functional, organized, and easy to understand.

Documentação

The repository must include a README.md file with clear instructions for running the solution.

The documentation must also explain the main technical decisions, including:

- solution overview;
- chosen architecture;
- backend organization;

- frontend organization;
- database choice;
- main trade-offs;
- testing strategy;
- known limitations;
- improvements that would be made with more time.

The documentation should be concise and objective. Its purpose is to help assess the candidate's technical reasoning.

Submission

The challenge must be submitted through a GitHub repository or an equivalent platform.

The repository must contain:

- backend source code;
- frontend source code;
- instructions for local execution;
- scripts or instructions for database setup;
- documentation of technical decisions;
- relevant automated tests.

Submissions in any other format will not be accepted.

Technical Presentation

After the submission is reviewed, the candidate may be invited to a technical interview to present the solution.

During the presentation, the candidate must explain:

- how the solution was structured;
- which technical decisions were made;
- what alternatives were considered;
- which trade-offs were accepted;
- how the solution could evolve;
- what they would improve with more time;
- how they would guide other developers working on the same solution;
- how they would conduct technical reviews and architecture discussions within the team.

The presentation is part of the evaluation process and will be used to assess the candidate's technical maturity, communication clarity, and technical leadership profile.

What Will Be Evaluated

The evaluation will consider the implementation, documentation, and technical presentation.

Evaluation Criteria

Criterion	What will be observed
Functional implementation	Whether the application implements the main required features
Solution architecture	Clear structure, separation of responsibilities, and ease of future evolution
Backend quality	API organization, modeling, persistence, error handling, and response consistency
Frontend quality	Application organization, API consumption, user flow, and interface clarity
Domain modeling	Correct representation of genres, authors, books, and their relationships
Automated tests	Relevant tests covering the main scenarios
Documentation	Clarity of instructions and explanation of technical decisions
Technical decisions	Ability to justify decisions and discuss alternative approaches
Tech Lead profile	Clear communication, critical thinking, ability to guide the team, and leadership in technical decision-making

Differentiators

The items below are not mandatory, but may positively contribute to the evaluation when applied coherently:

- containerization;
- integration tests;
- pagination, filters, or sorting;
- authentication or authorization;
- structured logs;
- global error handling;
- documentation of architectural decisions;
- simple validation pipeline;
- security awareness;
- observability awareness.

These items will not be evaluated as a checklist. What matters most is whether the choices are appropriate for the context of the challenge and clearly explained.

Deadline

The deadline for submitting the challenge is **3 calendar days** from the date the challenge is sent to the candidate.

After the submission review, candidates who receive a positive evaluation may be invited to the next stage: a technical interview in English to present the solution and discuss the decisions made.